

Lab 2: The IA64 Architecture And Its Implementation

This lab is based on the following documents: *Introducing the IA-64 Architecture*, *Itanium Processor Microarchitecture*, and *The Intel IA-64 Compiler Code Generator*. All of them can be found on the web page of the course.

1 Speculation

Question 1.1. What are the 3 main features of the IA-64 architecture for speculation? Describe each of them and explain why they are useful to achieve high performance. Note that such features may not explicitly be named "speculation" - speculation occurs any time the processor executes instructions which may not be necessary or useful to the program.

Answer.

- **Predication.** *Predication is a mechanism that allows any instruction to be executed speculatively and be graduated only if an associated predicate is true. This mechanism is particularly useful to better exploit the available parallel functional unit. Moreover, one can avoid simple if-then-else structures that traditionally involve the use of expensive branches. Using predication, both the if and else blocks can be executed in parallel once the processor has computed the predicate that tells which path to execute. Since the only instructions that will graduate are the one with a predicate whose value is true, only instructions along the correct path will change the state of the processor; the other ones (i.e. the ones that needn't have been executed) will be discarded. By speculatively executing blocks in parallel, one can avoid branch instructions and avoid their high cost. This mechanism is also used in software pipelining to minimise the overhead in term of code size generally associated with this technique (see the next question for details). Finally it allows more aggressive code scheduling technique and exposes more instruction level parallelism to the compiler by allowing optimisations to work on larger blocks.*
- **Control speculation.** *This mechanism allows to speculatively move a load instruction across basic block boundaries. It introduces a special `ld.s` instruction that perform the load but do not raise exceptions. A `chk.s` instruction is inserted by the compiler where the load is not speculative anymore (i.e. at its 'original' position). This instruction will branch to compiler-provided fix-up code if an exception occurred during the load. Since loads are really time consuming and often precede long calculation chains that depend on them, the earlier the load can be scheduled (by the compiler) the less instructions might wait because they depend on that load. Once more, this technique allows aggressive code scheduling techniques.*
- **Data speculation.** *This mechanism is somewhat similar to control specification. It allows the compiler to speculatively schedule a load instruction before store instructions that may access the same memory cell. For this purpose, the compiler uses advanced load instruction (`ld.a`) and the corresponding check instruction (`chk.a`) that will branch to fix-up code if necessary because of some store instruction modifying a loaded memory cell having been executed in between. As the two other techniques, it enables more aggressive code scheduling. Indeed a lots of programs use complex pointer-based data structures for which it is difficult (or even impossible) to get precise alias analysis, resulting in conservative decisions at compile-time. Typically, a load can hardly ever be moved before any stores at compile time without data speculation support in hardware.*

Question 1.2. What is the specific hardware support needed in order to use these features and that is implemented in Itanium?

Answer. For predication, 1-bit predicate registers are needed. Most instructions can then be associated to a predicate register to know whether they should graduate. Consequently, instruction format must include an (optional) predicate register field.

To support control speculation, all general purpose registers have an extra bit, called NaT. If an synchronous exception occurs due to the speculative load, the NaT bit of the target register is set and the exception is discarded. When the speculative check instruction is executed, it checks whether the NaT bit of the given register is set and if it is the case, it branches to fix-up code.

For data speculation, we need to keep track of the destination register, the address and the size of every advanced load. For that purpose, an advanced load address table (ALAT) is needed. When a `ld.a` occurs, a new line is added into the ALAT. When a store occurs and it accesses a position that has been loaded in advance (i.e. whose address overlap an entry in the ALAT), the corresponding line in the ALAT is erased. Finally, when a `chk.a` occurs, the fix-up code is executed if there are no corresponding lines in the ALAT (because a store erased it).

2 Software Pipelining and Register Model

Question 2.1. Describe the specific architectural support introduced in IA-64 for software pipelining. Explain why it makes software pipelining less expensive and easier.

Answer. Dedicated instructions for loop semantics are added, such as `br.ctop`. To support them, special registers for counting loop iterations (LC) and number of stages in the pipeline (EC) are provided. Moreover predicates are used to avoid duplicated code for the prologue and the epilogue. Finally, rotating registers (GP and predicates) are provided to support renaming among iterations: each rotation advances the pipeline by one stage.

Question 2.2. Considering the following program, write the assembly code that an Itanium compiler doing software pipelining would generate. Comment each line. The Itanium instruction set can be found in Intel Itanium Architecture Software Developer's Manual Volume 3.

```
int *ptr1, *ptr2, i;
ptr1 = R1;
ptr2 = R2;

for (i = 0; i < 100; ++i) {
    *ptr2 = *ptr1;
    ++ptr1;
    ++ptr2;
}
```

<code>mov LC = 99</code>	Initialise the loop counter
<code>mov EC = 3</code>	Initialise the epilogue counter
<code>mov pr.rot = 1 << 16</code>	Initialise the rotating predicate registers

Answer.	<code>loop:</code>	
	<code>(p16) ld4 R34 = [R1], 4</code>	Stage 1: load *ptr1 <i>Nothing in stage 2 (p17) because of the load latency (2 cycles)</i>
	<code>(p18) st4 [R2] = R36, 4</code>	Stage 3: store to ptr2
	<code>br.ctop loop;;</code>	Redo the loop

Note that we (as Intel's compiler does) have not used a rotating register for the addresses.

Question 2.3. Suppose that the for loop is replaced by a while loop, for example:

```
while (*ptr1 != NULL)
```

Explain what kind of changes your code would need.

Answer. In the case of a while loop, the LC register and the associated `br .c top` instruction cannot be used anymore because we cannot know at compile time the number of iteration of the loop.

Consequently one should use the `br .w top` instruction that allows to branch only if a predicate has the value true. The result of the comparison of the while statement must be put in a predicate register that will be used by the `br .w top` instruction. That predicate register must be a one of the rotating predicate registers in order to be able to drain the loop when the loop condition is not true anymore.

Moreover, the `ld` should be made speculative because at the time the condition becomes false, we will have already executed the load in a next iteration (since the test can only occur in late pipeline stages).

Question 2.4. Explain the register model for the programmer and the parameter passing convention. Illustrate the concept with the code which calls a C function:

```
int foo(int a, int b, int c)
```

List advantages and disadvantages of this scheme in terms of

- (1) programmer/compiler comfort,
- (2) microarchitectural complexity
- (3) performance.

Indicate with a simple diagram how the register addressing scheme might be implemented.

Answer. Infinitely many registers are provided to the programmer through the stack engine. Beside the first 32 registers, 96 registers are managed as a stack. The register stack engine (RSE) saves registers to memory when not enough free register are available. Each procedure can allocate up to 96 fresh registers through `alloc` instructions and deallocate them on procedure return.

The interest of such a model is that it results in procedure calls being less expensive by avoiding systematic saving of registers on the (memory) stack. It simplifies the task of the compiler because of the virtually infinite number of registers.

Finally, the RSE can be implemented so as to speculatively spill and fill registers in the background using unused memory ports.

3 VLIW Processor Scheduler Simulation

Consider now a simplified processor, inspired by the Explicitly Parallel Instruction Computing (EPIC) design style, similar to the Intel IA-64 (Itanium) architecture. This processor executes a simplified RISC assembly.

In the following exercise, you will act as the compiler, applying various techniques described below to schedule, optimize, and pipeline a simple loop onto this simple VLIW processor.

Consider the following pseudocode along with its equivalent assembly code.

High-level code snippet

```
for i in range(17):  
    listB[i] = listA[i] + listA[(i + 1) % 17]
```

Equivalent assembly code

```
00 mov LC, 16  
01 mov x2, 0x1000  
02 mov x3, 0x2000  
03 ld x4, 0(x2)  
04 ld x5, 1(x2)  
05 add x6, x4, x5  
06 st x6, 0(x3)  
07 addi x2, x2, 1  
08 addi x3, x3, 1  
09 loop 03  
10 mov x7, 0x2000  
11 ld x2, 0(x3)  
12 ld x4, 0(x7)  
13 add x6, x2, x4  
14 st x6, 0(x3)
```

The processor has two ALUs, which can execute any of the arithmetic instructions (add, addi, and mov), a memory unit (ld and st instructions) which can execute the load and store operations, and a branch unit able to execute the loop instruction.

add and addi have a latency of 2, memory operations have a latency of 2, loop and mov operations have a latency of 1.

The loop X operation corresponds to the following pseudocode (LC is a register):

```
if (LC > 0):  
    LC = LC - 1  
    jump to instruction X
```

The processor has 96 general-purpose registers named x0 to x95. There are the same number of predicate registers (named p0 to p95). Registers 0 to 31 are static, and the other ones are rotating. There are also two special registers: LC (Loop Counter) and EC (Epilogue Count). Rotating registers, predicates and EC will be useful at the end of the exercise, when you will pipeline the loop.

Note also that in this VLIW processor, memory is word-addressable (i.e., address i and address $i + 1$ point to different words).

The tables that you have to fill can be downloaded on Moodle. All the tables are in the same file, but on separated sheets. In the end, the goal is to schedule all the instructions statically, as efficiently as possible.

Question 3.1. Fill in the gaps in the dependency table (Question 1 sheet). There are two main kinds of dependencies:

1. True dependencies (Read after Write dependencies): These are the data dependencies that cannot be solved by allocating more registers. These dependencies can be separated in 4 different types:

- (a) Local: if *instruction_2* depends locally on *instruction_1*, then both instructions are before the loop, or both instructions are after the loop, or both instructions are in the loop but *instruction_2* comes after *instruction_1* (the dependency stays in the iteration)
 - (b) Interloop: if *instruction_2* depends on *instruction_1* but *instruction_2* comes before *instruction_1*, then the dependency is carried from one iteration to another. Often, *instruction_2* also depends on an instruction that happens before the loop (for the initialization step) (in the table, mark both of them)
 - (c) Loop Invariant: if *instruction_2* depends on *instruction_1* and *instruction_2* is in the loop but *instruction_1* is before the start of the loop, then this dependency only happens once (at the first iteration of the loop), and thus these dependencies can be handled before the start of the loop.
 - (d) Post Loop: if *instruction_2* depends on *instruction_1* and *instruction_2* is after the loop but *instruction_1* is in the loop, then this dependency can be handled after the end of the loop.
2. False dependencies: These are all dependencies that can be solved with appropriate register allocation. There are 2 different types :
- (a) Anti dependencies (Write after Read dependency): if *instruction_2* writes in *x1* but *instruction_1* reads from *x1*, then putting *instruction_2* before *instruction_1* causes *instruction_1* to read the wrong value.
 - (b) Output dependencies (Write after Write dependency): if both *instruction_2* and *instruction_1* write in *x1*, then reordering these instructions causes *x1* to hold the wrong value.

In the case where a dependency on *A* is spread across multiple iterations, note *A'* to indicate this dependency.

Question 3.2. Fill in the gaps in the dependency table (Question 1 sheet). Schedule statically these instructions, by filling in the gaps in the template (Question 2 sheet). When no instruction can be placed, place an empty instruction (i.e., nop). Take into account only the true dependencies and the latency of operations, and use ASAP scheduling. You will have to update the loop operation.

Every line of this table is called a **bundle**: the set of instructions that are executed at the same time.

Note: Do not worry about loop pipelining and register renaming (as new registers are allocated to handle false dependencies)

Question 3.3. Handle the remaining dependencies by applying register renaming (Question 3 sheet). The procedure is the following:

1. Allocate a register (starting from *x1*, *x2*, ...) to each destination register (even when there is an inter-loop dependency). Update correctly the operands of each instruction. (Fill the destination register column from the dependency table with the new registers, except for LC)
2. For each interloop dependencies that have a dependency on an instruction that executes before the loop, add a mov instruction at the end of the loop bundles to move the value from the second destination register (the one in the loop) to the first one (the one outside the loop). If needed, add a new loop bundle to add these mov instructions. *Note : The added mov instructions are placed in the last available bundle of the loop for simplicity.*
3. Update correctly operands registers in the dependency table.

*Note: Since there is no loop pipelining, you only have to use static registers (*x0* – *x31*).*

Question 3.4. The processor starts a new loop iteration every *II* cycles (Initiation Interval). The goal is to find the smallest *II* such that the loop can be scheduled.

Compute the lower bound : for each unit type (ALU, Memory, Branch), count how many operations of that type appear in one loop iteration. If a unit type has *U* identical units and the loop body contains *N* operations of that type, then that unit type imposes:

$$II \geq \left\lceil \frac{N}{U} \right\rceil$$

Take the maximum over all unit types.

Question 3.5. Now, instead of executing one iteration completely before starting the next one, execute several iterations of the loop simultaneously.

The pipelined loop is divided into stages. Each stage corresponds to a group of instructions belonging to one logical iteration of the loop. When pipelining is active, different stages from different iterations are executed in parallel. At every II cycles, a new iteration enters the pipeline. The processor therefore overlaps several iterations.

For this exercise, we only give you procedure you can simply apply to understand how loop pipelining works. If you want a more detailed explanation for loop pipelining, you can read the sections 3.1 *Initiation Interval* and 3.2.2 *Scheduling With loop.pip* of Homework 2.

Complete the Question 5 sheet using the following steps:

1. Instructions before and after the loop are pipelined the same way as before.
2. Using the previously computed II , create the first stage with II bundles. Every stage will contain II bundles.
3. Place each loop instruction in one of the bundles while respecting the processor constraints.

For each instruction:

- (a) Select a bundle (from 0 to $II - 1$) where the instruction could be executed, in a stage where the local and loop invariant dependencies are respected. If this stage doesn't exist, create it by adding a new stage of II bundles.
 - (b) Mark the unit in all corresponding bundles in other stages as "reserved" (mark it as --): no other instruction can be placed there (as the unit will be used for this instruction).
 - (c) Verify the latency constraint: if the interloop dependencies cannot be satisfied, then the stage is too small (II is too small). Start the scheduling again, and choose II to be the number of stages needed to satisfy this dependency.
4. The loop instruction should be placed at the end of the first stage, and it should jump at the start of the same stage.

Question 3.6. If the same architectural registers were reused for every iteration, instructions from different iterations would overwrite each other's values.

To avoid these conflicts, the processor provides rotating registers: registers $x32$ to $x95$.

Each time II bundles are executed (the current stages are finished), the rotating register window is shifted. This means that the same logical register name in the program refers to a different physical register at each iteration. This allows different iterations of the loop to keep their own values without interfering with each other. Here is how they are defined:

$$xZ(+a, +b)$$

where:

- Z is the base rotating register,
- a is the iteration offset,
- b is the stage offset.

The resulting register is therefore equivalent to:

$$x(Z + a + b)$$

Complete the Question 6 sheet using the following steps:

1. Determine the number of stages of the pipelined loop from the previous question.

2. Allocate rotating registers to all instructions inside the loop that produce a value.

Registers are allocated starting from x32. To avoid conflicts between overlapping iterations, allocate one register every (#stages + 1) registers (for example: x32, x36, x40, ... if there are 3 stages).

3. Allocate static registers for loop invariant dependencies and local dependencies outside the loop. Static registers start from x1.

4. For each instruction inside the loop, update the source operands using the notation:

$$xZ(+a, +b)$$

where:

- a represents the iteration offset (0 for the current iteration, 1 for the next iteration),
- b represents the difference between the stage of the consumer instruction and the stage of the producer instruction.

5. Replace the notation $xZ(+a, +b)$ by the corresponding physical register $x(Z + a + b)$.

6. For post loop dependencies, the register is produced in the last iteration of the loop (iteration offset is 0, stage offset is the stage distance between the producer and last stage of the loop).

Answer. See annex.